Pokročilé objektové technológie

prednáška

7



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

GUI – Model View ViewModel

prof. Ing. **Emil Kršák**, PhD.

KST Katedra softvérových technológií
Fakulta riadenia a informatiky, Žilinská univerzita v Žiline

Čo je MVVM

- Model View ViewModel
- Architektonický štýl
- Špecializácia „Presentation Model design pattern“
- Vychádza z MVC a Presentation Model (M. Fowler), MVVM: J. Gossman, Microsoft, 2005
- Vhodný pre XAML technológie s data bindingom: WPF, WinUI 3, .NET MAUI, Avalonia

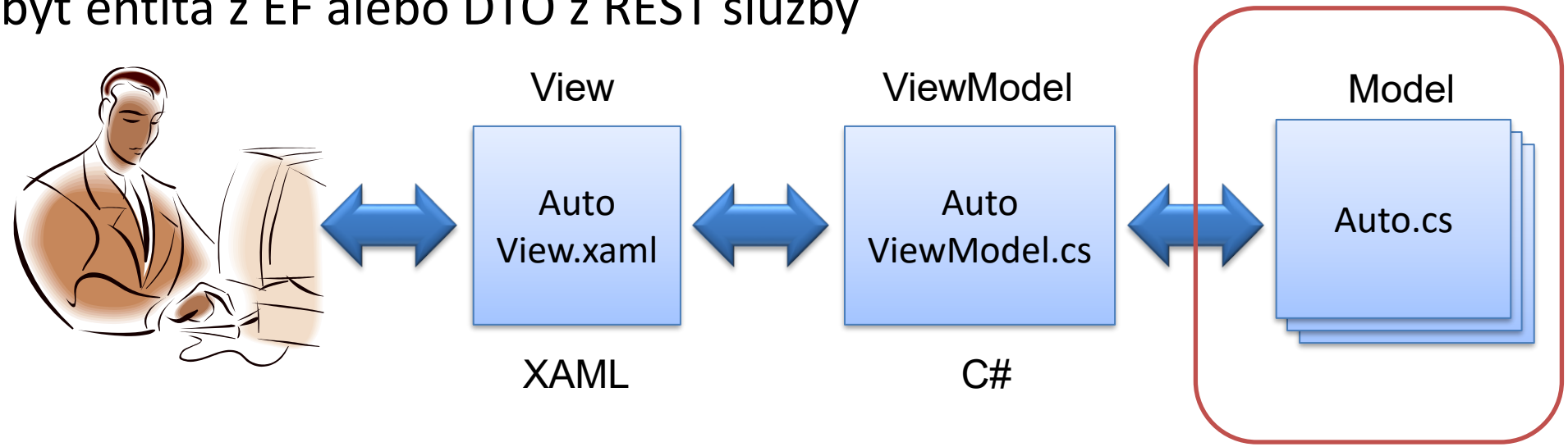
Čo znamená MVVM



Každá vrstva pozná iba vrstvu vpravo od seba: View → ViewModel → Model

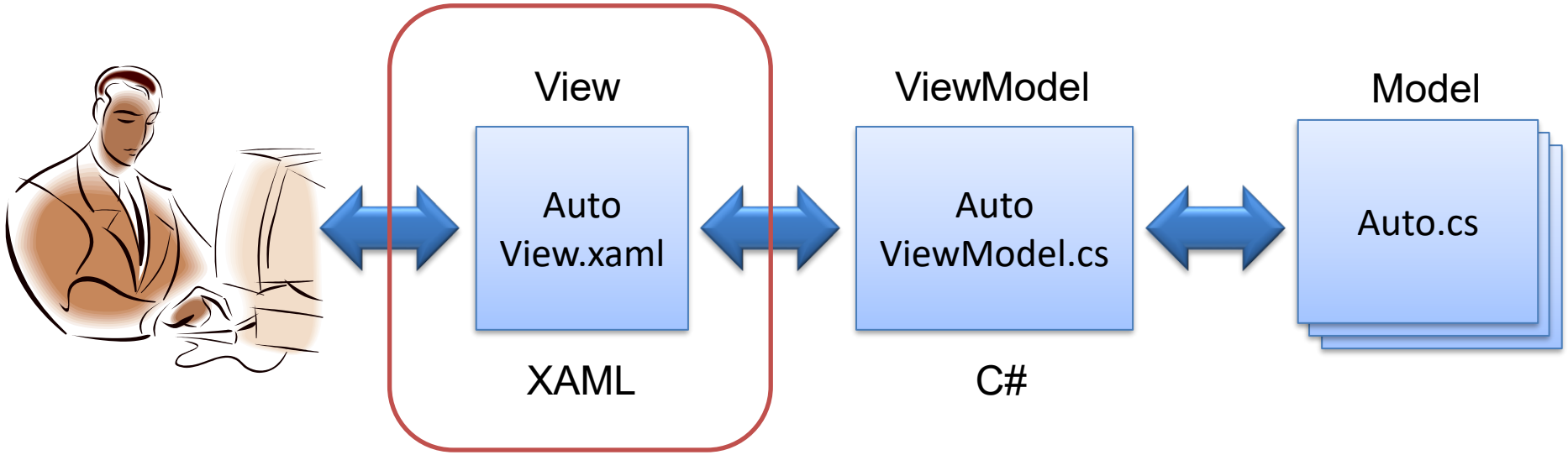
Čo je Model

- Dátový model – podľa typu aplikácie
  - Objektový model aplikácie (object-oriented approach)
  - Relačný model aplikácie (data-centric approach)
- Nevie nič o existencii View a ViewModel
- Komunikuje s ViewModelom
  - Prijíma správy (volania metód)
  - Notifikuje ViewModel pomocou udalostí (event) alebo INotifyPropertyChanged.PropertyChanged a INotifyCollectionChanged.CollectionChanged
- Môže byť entita z EF alebo DTO z REST služby



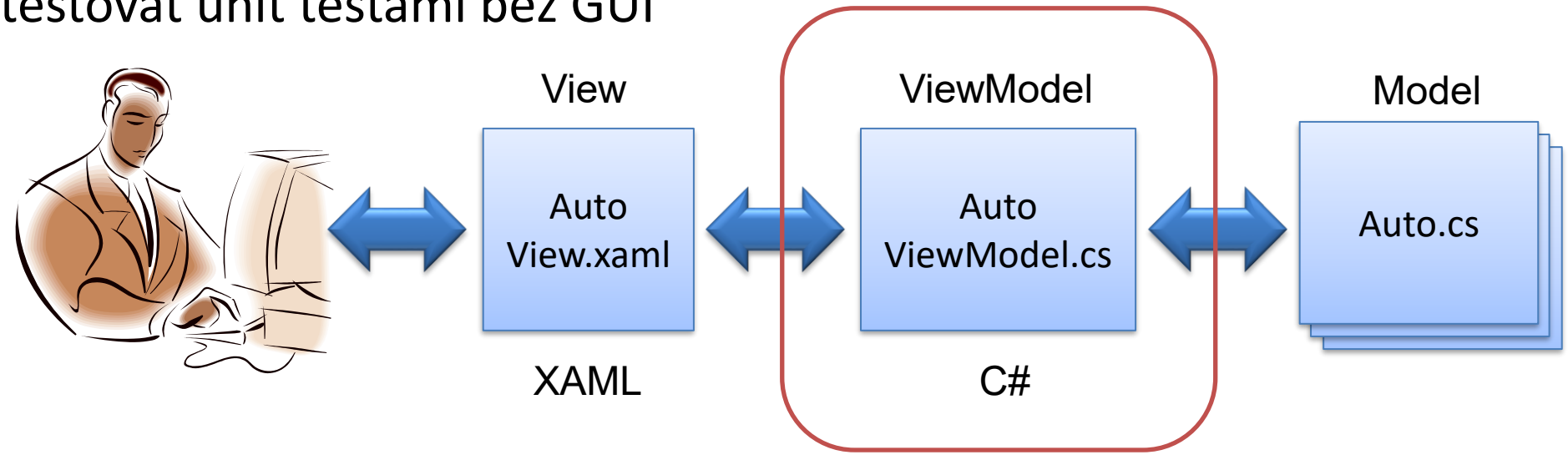
Čo je View

- Elementy používateľského rozhrania
- Okná, tlačidlá, zoznamy, grafika, ...
- Komunikuje iba s ViewModelom – data binding, príkazy (ICommand), správy
- Nemusí poznať štruktúru Modelu
- Často sa využíva reflexia pre akcie a previazanie DataBinding
- Všetko iba v XAML, minimum code-behind

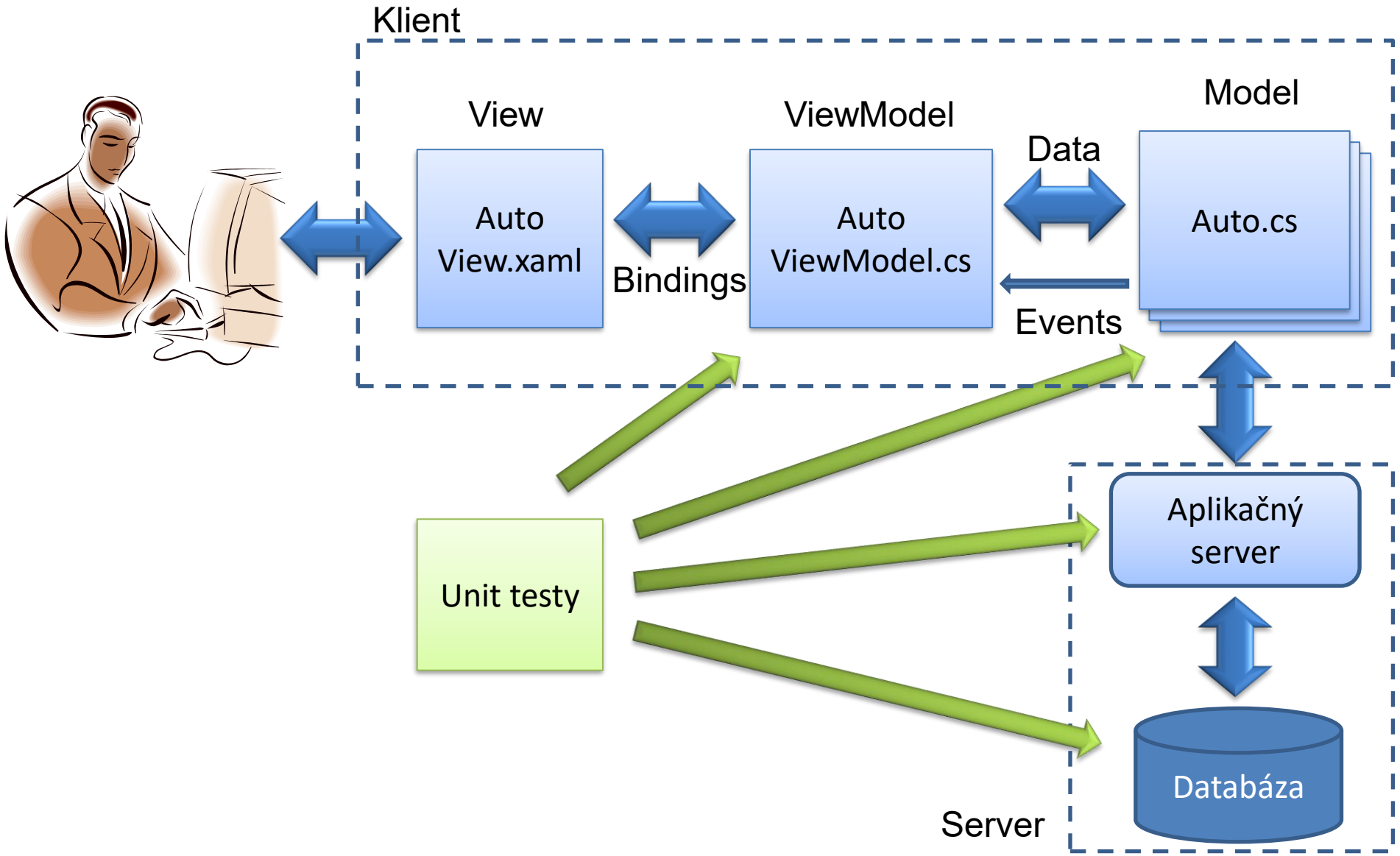


Čo je View-Model

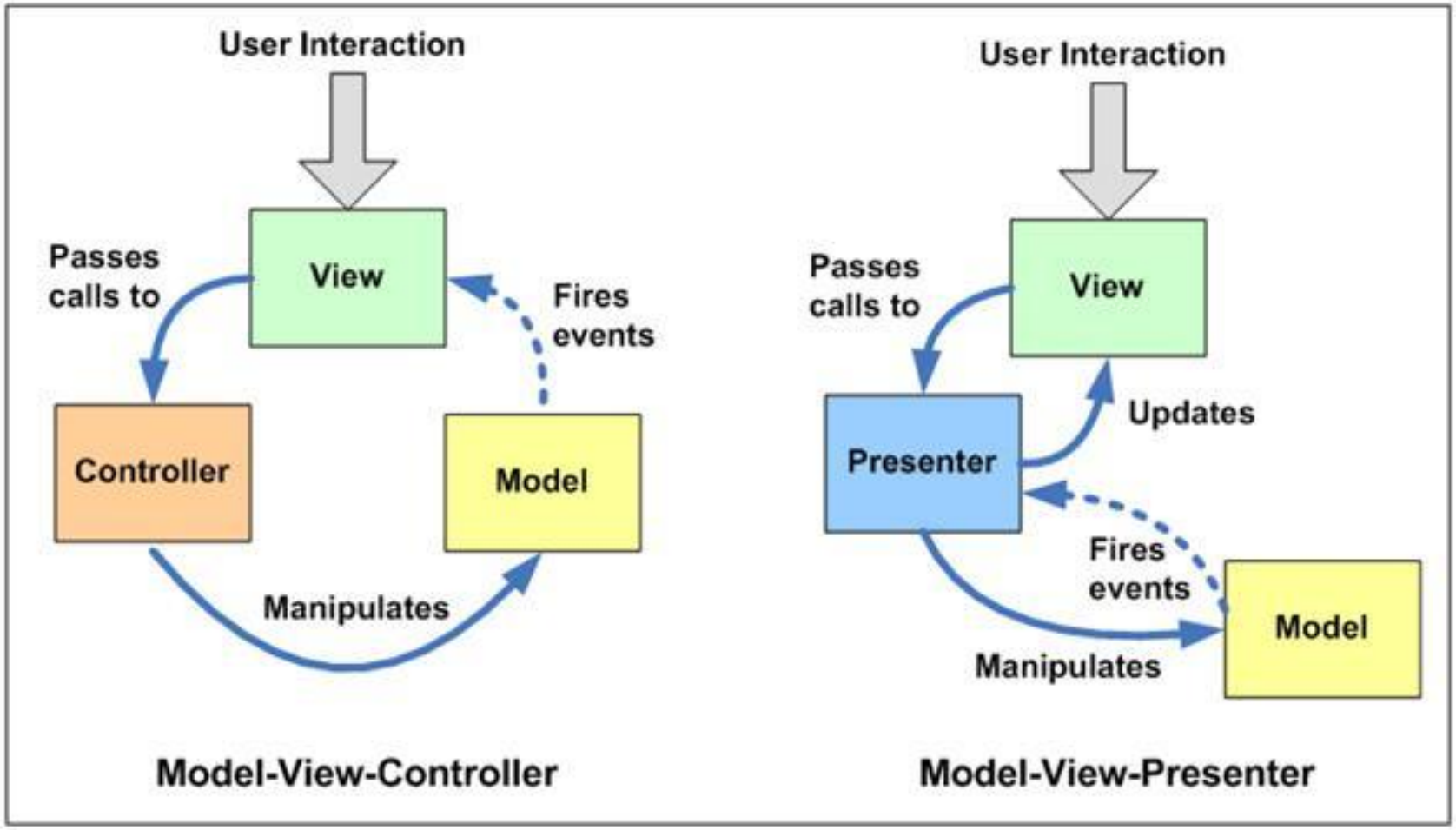
- Je model pohľadu – abstrakcia View
- Nepozná štruktúru View
- Neposiela dáta do View – View si ich vyžiada cez binding na property
- Pre View pripravuje public property s notifikáciou (INotifyPropertyChanged) a príkazy (ICommand)
- Robí transformáciu v reálnom čase medzi dátovým modelom (Model) a modelom GUI pre View
- Transparentne zabezpečuje DataBinding medzi Model a View
- Dá sa testovať unit testami bez GUI



Ako sú prepojené?

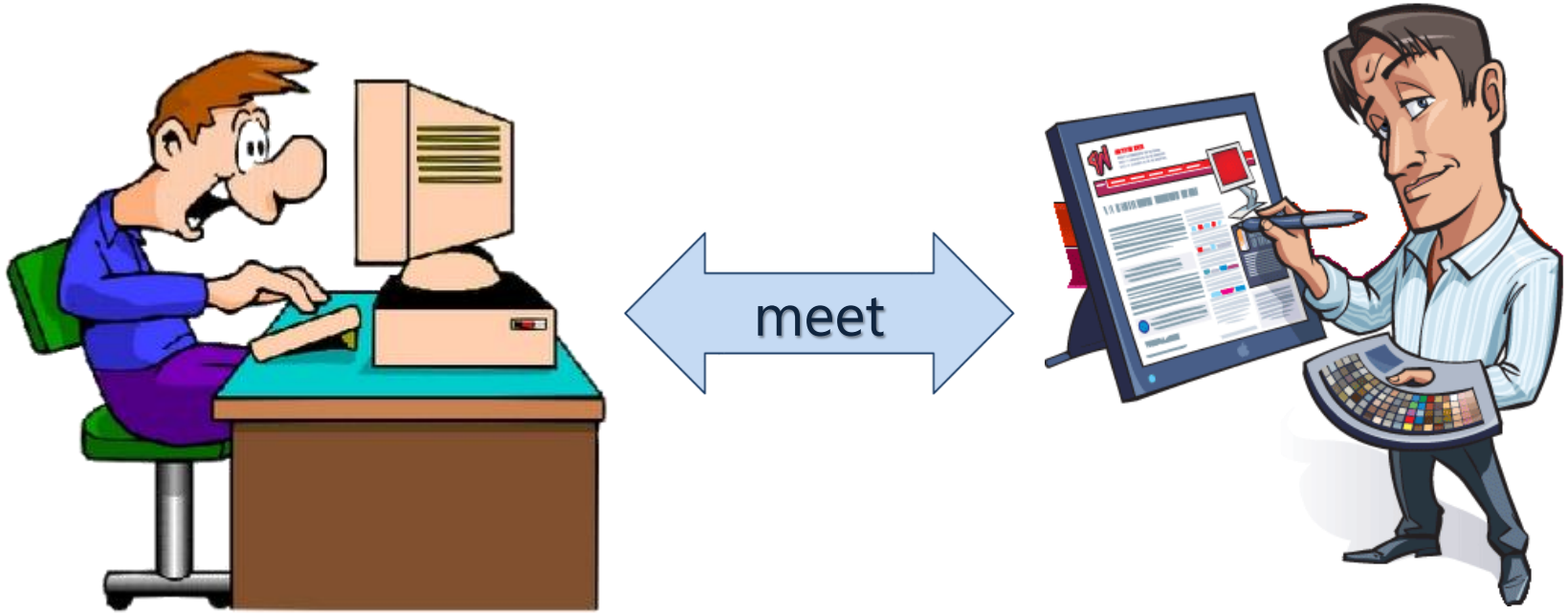


Porovnanie s MVC a MVP



Oddelenie dizajnéra od programátora

Developers are not the only ones responsible for their applications!



Data binding

WPF Data Binding

```
<TextBox Text="{Binding City,
    UpdateSourceTrigger=PropertyChanged}" />
```

```
class MainWindowViewModel : INotifyPropertyChanged
{
  private string city = "";
  public string City
  {
    get => city;
    set { city = value; OnPropertyChanged(); }
  }

  public event PropertyChangedEventHandler? PropertyChanged;

  void OnPropertyChanged([CallerMemberName] string? n = null)
    => PropertyChanged?.Invoke(this, new(n));
}

// v konštruktore okna (code-behind):
DataContext = new MainWindowViewModel();
```

binding



Data binding

- Zdroj (Source) bindingu môže byť určený rôznymi spôsobmi
- DataContext sa dedí – nastaví sa na okne na ViewModel a platí pre všetky vnorené prvky
- Potom:
  - {Binding Prop}, {Binding Prop.SubProp}
  - {Binding Prop, Mode=TwoWay, UpdateSourceTrigger=PropertyChanged}
  - {Binding Prop, Converter={StaticResource x}}
  - {Binding Prop, ElementName=x}
  - {Binding Prop, RelativeSource={RelativeSource AncestorType=Window}}

Ako povie VM, aby View niečo urobilo

- Notifikáciami
  - INotifyPropertyChanged.PropertyChanged

```
OnPropertyChanged("City");
OnPropertyChanged(nameof(City));
OnPropertyChanged();   // [CallerMemberName]
```

  - INotifyCollectionChanged.CollectionChanged (ObservableCollection<T>)
- View je zavesený na udalosti VM
- Správami (messenger) – VM nepozná príjemcu
- VM volá View cez interface – používať opatrne (napr. IDialogService)
- Použitím custom binding extensions

Ako tvoriť aplikáciu

- View first
  - Začína sa s View, ktorý má presne definovaný VM
  - Klasické aplikácie
- ViewModel first
  - Aplikácia pracuje s VM
  - Príslušný View sa vyhľadá (DI kontajner, DataTemplate podľa typu VM, konvencie)
  - Vhodné pre multiplatformové aplikácie (viac možných View dynamicky linkované)

MVVM frameworky

CommunityToolkit.Mvvm

Microsoft / .NET Foundation

Ľahký, source generátory ([ObservableProperty], [RelayCommand]). Odporúčaný začiatok.

aktívny

Prism

Prism Library (pôv. Microsoft p&p)

Navigácia, moduly, DI, dialógy – pre veľké aplikácie. Od verzie 9 duálna licencia.

aktívny

Caliburn.Micro

Rob Eisenberg, komunita

Konvencie namiesto konfigurácie (x:Name ↔ property). Ukážky v tejto prednáške.

udržiavaný

ReactiveUI

Paul Betts, komunita

Reaktívne MVVM nad Reactive Extensions (Rx).

aktívny

Catel, WAF

komunita

Ďalšie dlhodobo udržiavané frameworky pre WPF.

aktívne

MVVM Light

Laurent Bugnion

Kedysi najpopulárnejší, vývoj ukončený (2021). Nástupca: CommunityToolkit.Mvvm.

ukončený

Caliburn.Micro – ako začať

Postup

1. Nový projekt WPF App (.NET 10)
2. NuGet balík Caliburn.Micro (5.x)
3. V App.xaml zmazať StartupUri, odstrániť MainWindow.*
4. Pridať Bootstrapper, ShellViewModel a ShellView

Konvencia

ShellViewModel → ShellView

Caliburn.Micro nájde View podľa mena ViewModelu

App.xaml.cs

```
public partial class App : Application
{
    public App()
    {
        var bootstrapper = new Bootstrapper();
    }
}
```

Bootstrapper.cs

```
public class Bootstrapper : BootstrapperBase
{
    public Bootstrapper() => Initialize();

    protected override async void OnStartup(object sender, StartupEventArgs e)
        => await DisplayRootViewForAsync<ShellViewModel>();
}
```

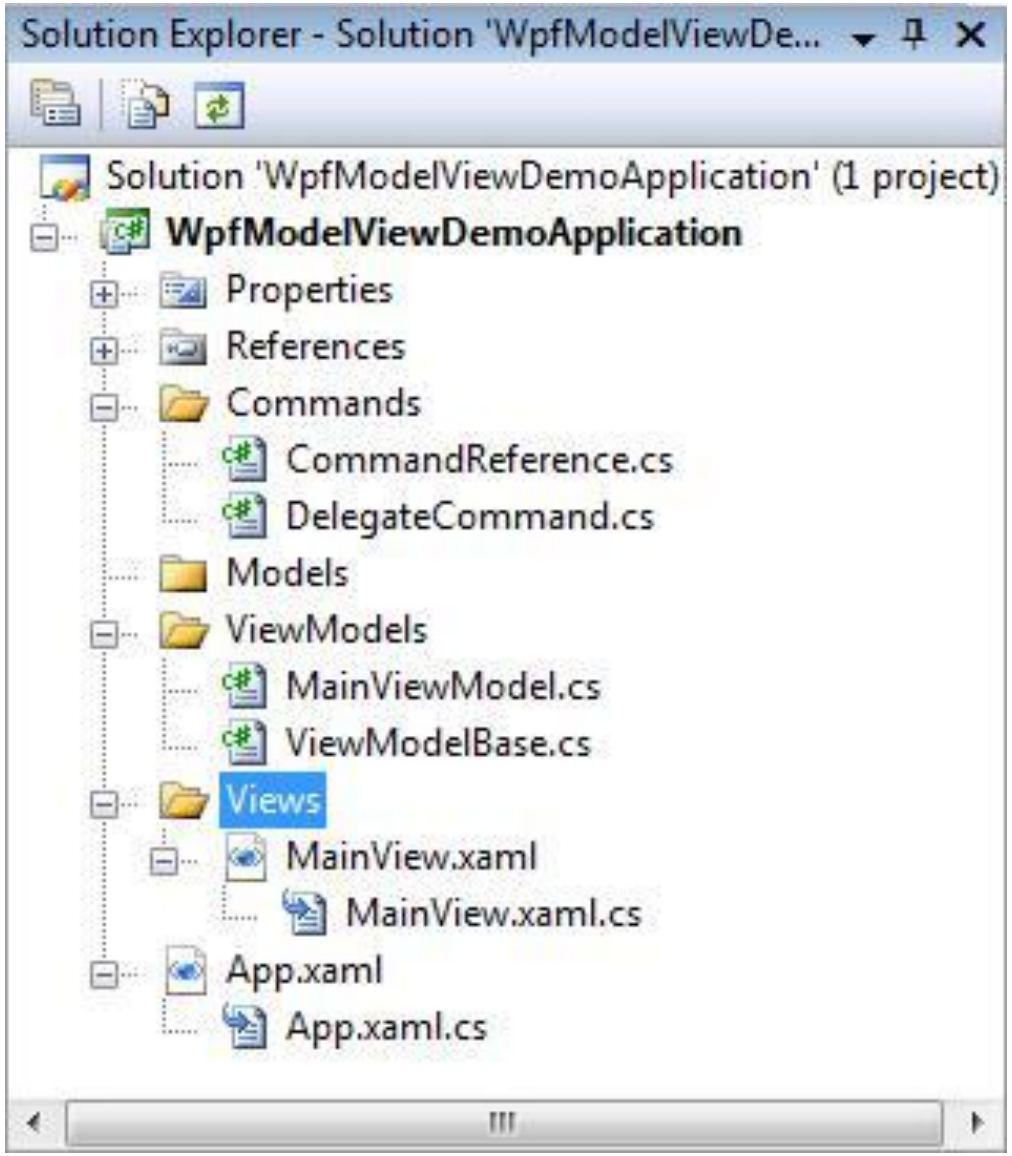
ShellViewModel.cs

```
public class ShellViewModel : Screen { }
```

ShellView.xaml

```
<Window x:Class="MvvmUkazka.ShellView" xmlns="..." xmlns:x="..."
        Title="Ukážka MVVM" Width="400" Height="200">
  <TextBlock Text="Hello Caliburn Micro!" FontSize="20"
             HorizontalAlignment="Center" VerticalAlignment="Center" />
</Window>
```

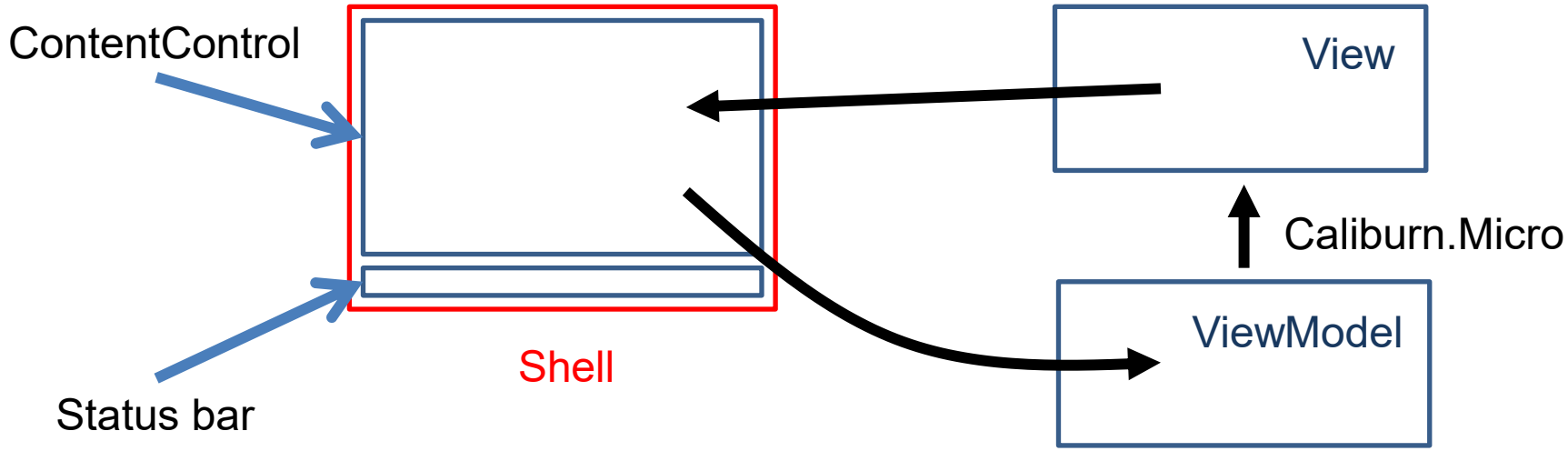
Rozloženie súborov v MVVM



Čo je shell

- Väčšinou hlavné prázdne okno
- Obsah
  - ContentControl
    - Binding na ActiveItem (ShellViewModel : Conductor<object>)
    - View sa k tomu dohľadá automaticky na základe konvencií
  - Status bar
  - Príp. ďalšie spoločné prvky pre celú aplikáciu

```
<ContentControl x:Name="ActiveItem" Focusable="False"/>
```



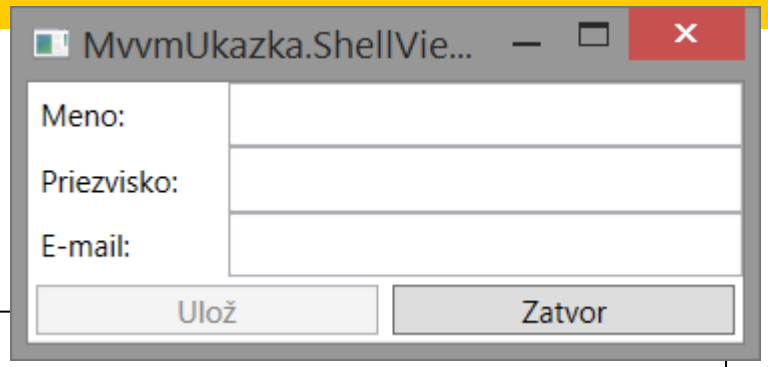
Konvencie v Caliburn.Micro

```
public class ShellViewModel : Screen
{
    private string _meno = "", _prie = "", _mail = "";

    public string TxtMeno
    {
        get => _meno;
        set { Set(ref _meno, value); Zmena()
    }
    public string TxtPrie
    {
        get => _prie;
        set { Set(ref _prie, value); Zmena()
    }
    public string TxtMail
    {
        get => _mail;
        set { Set(ref _mail, value); Zmena()
    }
    private void Zmena() => NotifyOfProperty

    public bool CanCmdSave =>
        !string.IsNullOrWhiteSpace(TxtMeno)
        !string.IsNullOrWhiteSpace(TxtPrie)
        TxtMail.Contains('@');

    public void CmdSave() => MessageBox.Show
    public Task CmdExit() => TryCloseAsync()
}
```




```
<Grid>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="80" />
        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>
    <Grid.RowDefinitions>
        <RowDefinition/>
        <RowDefinition/>
        <RowDefinition/>
        <RowDefinition/>
    </Grid.RowDefinitions>
    <Label Content="Meno:" />
    <Label Grid.Row="1" Content="Priezvisko:" />
    <Label Grid.Row="2" Content="E-mail:" />
    <TextBox Grid.Row="0" Grid.Column="1" x:Name="TxtMeno" />
    <TextBox Grid.Row="1" Grid.Column="1" x:Name="TxtPrie" />
    <TextBox Grid.Row="2" Grid.Column="1" x:Name="TxtMail" />
    <UniformGrid Grid.Row="3" Rows="1" Grid.ColumnSpan="2">
        <Button Content="Ulož" x:Name="CmdSave" Margin="3"/>
        <Button Content="Zatvor" x:Name="CmdExit" Margin="3"/>
    </UniformGrid>
</Grid>
```

ViewModel bez MessageBox – IDialogService

IDialogService.cs — *rozhranie – nepozná WPF*

```
public interface IDialogService
{
    void ShowMessage(string text, string title = "");
    bool Confirm(string text, string title = "");
}
```

DialogService.cs — *implementácia – vrstva View*

```
public class DialogService : IDialogService
{
    public void ShowMessage(string text, string title = "")
        => MessageBox.Show(text, title);

    public bool Confirm(string text, string title = "")
        => MessageBox.Show(text, title, MessageBoxButton.YesNo)
           == MessageBoxResult.Yes;
}
```

Prečo?

- ViewModel závisí iba od rozhrania – nepozná WPF ani MessageBox
- Kontajner dosadí DialogService do konštruktora (Dependency Injection)
- V unit teste sa podstrčí FakeDialogs – test beží bez okna

ShellViewModel.cs — *primárny konštruktor*

```
public class ShellViewModel(IDialogService dialogs) : Screen
{
    // ... TxtMeno, TxtPrie, TxtMail, CanCmdSave ...

    public void CmdSave()
    {
        if (dialogs.Confirm($"Uložiť {TxtMeno} {TxtPrie}?"))
            dialogs.ShowMessage("Uložené.");
    }
}
```

Bootstrapper.cs — *SimpleContainer*

```
protected override void Configure()
{
    _container.Singleton<IWindowManager, WindowManager>();
    _container.Singleton<IDialogService, DialogService>();
    _container.PerRequest<ShellViewModel>();
}
```

Bootstrapper ešte prepíše GetInstance, GetAllInstances a BuildUp tak, aby volali _container.

CommunityToolkit.Mvvm – ten istý príklad

```
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Input;

public partial class ShellViewModel : ObservableObject
{
    [ObservableProperty]
    [NotifyCanExecuteChangedFor(nameof(UlozCommand))]
    private string meno = "";

    [ObservableProperty]
    [NotifyCanExecuteChangedFor(nameof(UlozCommand))]
    private string priezvisko = "";

    [ObservableProperty]
    [NotifyCanExecuteChangedFor(nameof(UlozCommand))]
    private string mail = "";

    private bool MozemUlozit() =>
        Meno.Length > 0 && Priezvisko.Length > 0 && Mail.Contains('@');

    [RelayCommand(CanExecute = nameof(MozemUlozit))]
    private void Uloz() { /* uloženie */ }
}
```

[ObservableProperty]
pole meno → property Meno s PropertyChanged

[NotifyCanExecuteChangedFor]
zmena hodnoty prepočíta, či sa dá uložiť

[RelayCommand]
metóda Uloz() → príkaz UlozCommand (ICommand)

CanExecute
false → tlačidlo je automaticky zakázané

partial
zvyšok kódu vygeneruje kompilátor (source generator)

```
<TextBox Text="{Binding Meno, UpdateSourceTrigger=PropertyChanged}" />
<TextBox Text="{Binding Priezvisko, UpdateSourceTrigger=PropertyChanged}" />
<TextBox Text="{Binding Mail, UpdateSourceTrigger=PropertyChanged}" />
<Button Content="Ulož" Command="{Binding UlozCommand}" />
```

Zásady pri tvorbe aplikácie MVVM

- Minimalizovanie code-behind
- Bindovať všetky vstupy/výstupy cez ViewModel
  - Bindovať priamo na Model prináša v budúcnosti problém
- Implementovať INotifyPropertyChanged vo ViewModeli
- Akcie cez príkazy (ICommand) namiesto Click v code-behind
- ViewModel nepozná View – žiadne MessageBox ani Window, použiť služby (napr. IDialogService)
- ViewModel by mal eliminovať problém s viacvláknovosťou Modelu
- Unit testy tvoriť pre ViewModel (ViewModel brať ako reálne UI)



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Katedra softvérových
technológií

emil.krsak@uniza.sk

ĎAKUJEM ZA POZORNOSŤ

Upozornenie

- Tieto študijné materiály sú určené výhradne pre študentov Fakulty riadenia a informatiky Žilinskej univerzity v Žiline

- Reprodukovanie, šírenie (i častí) materiálov bez písomného súhlasu autora nie je dovolené



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

prof. Ing. **Emil Kršák**, PhD.

emil.krsak@uniza.sk